

```
"""
Real provider adapters, matching each
SDK's actual current interface.
These CANNOT be exercised in this
sandbox -- no network, no SDK install
possible here. They're written to be
correct and ready, not simulated.
```

```
Requires (once you have network +
keys):
```

```
    pip install anthropic openai
    export ANTHROPIC_API_KEY=...
    export OPENAI_API_KEY=...
    export XAI_API_KEY=...
```

```
"""
```

```
import os, time, json
from kernel import ModelAdapter,
ModelRequest, ModelResponse
```

```
class ClaudeAdapter(ModelAdapter):
```

```
    name = "claude"
```

```
    def __init__(self, model="claude-
sonnet-5", api_key=None):
        import anthropic
        self.client =
anthropic.Anthropic(api_key=api_key or
```

```

os.environ.get("ANTHROPIC_API_KEY"))
    self.model = model

    def generate(self, request:
ModelRequest) -> ModelResponse:
        start = time.time()
        system_prompt = (
            "You are an operator over a
persistent relational substrate
(PRCSA). "
            "You will receive a task
and a projected workspace (a bounded
slice of "
            "current state). Respond
with plain reasoning, then end with a
single "
            "JSON block on its own line
describing ONE candidate structural "
            "contribution, in exactly
this shape:\n"
            '{"content": "...",
"scope": {...}, "evidence_class":
"REAL|SIMULATED|'
            'PREDICTED|INTERVENTION_DERIVED|INFERRE
D", "kind": "CLAIM|HYPOTHESIS|'
            'PREDICTION|QUESTION|SIMULATION|ACTION_
PROPOSAL|INTERPRETATION|'
            'UNCERTAINTY|TRANSLATION|HOLD",

```

```

"falsification_test": null}\n'
        "If you do not have enough
support to commit to a claim, respond
with "
            '{"kind": "HOLD",
"content": "why"} instead -- this is a
legitimate, '
            "preferred outcome, not a
failure. Do not invent a CLAIM just to
"

            "produce output. If
evidence_class is INFERRED,
falsification_test "
            "MUST be a concrete,
checkable prediction, not null. You do
not have "

            "write access to the
substrate -- the gate decides what
happens with "
            "this candidate."
        )
        resp =
self.client.messages.create(
            model=self.model,

max_tokens=request.max_tokens,
            system=system_prompt,
            messages=[{"role": "user",
"content": (
                f"TASK:
{request.task}]\n\nPROJECTED

```

```

WORKSPACE:\n"
        f"
{json.dumps(request.projection,
indent=2)}"
        )}],
    )
    latency = (time.time() - start)
* 1000
    raw_text = resp.content[0].text
    candidate =
_extract_json_candidate(raw_text)
    return ModelResponse(
        raw_text=raw_text,
model_id=self.model,
latency_ms=latency,
        token_usage={"input":
resp.usage.input_tokens, "output":
resp.usage.output_tokens},
        candidate=candidate,
    )

class GPTAdapter(ModelAdapter):
    name = "gpt"

    def __init__(self, model="gpt-5",
api_key=None):
        import openai
        self.client =
openai.OpenAI(api_key=api_key or
os.environ.get("OPENAI_API_KEY"))

```

```

        self.model = model

        def generate(self, request:
ModelRequest) -> ModelResponse:
            start = time.time()
            system_prompt = (
                "You are an operator over a
persistent relational substrate
(PRCSA). "
                "Respond with reasoning,
then a single JSON block: "
                '{"content": "...",
"scope": {...}, "evidence_class":
"REAL|SIMULATED|'

                'PREDICTED|INTERVENTION_DERIVED|INFERRE
D", "kind": "CLAIM|HYPOTHESIS|'

                'PREDICTION|QUESTION|SIMULATION|ACTION_
PROPOSAL|INTERPRETATION|'

                'UNCERTAINTY|TRANSLATION|HOLD",
"falsification_test": null}. '
                "If you do not have enough
support to commit to a claim, respond "
                'with {"kind": "HOLD",
"content": "why"} instead -- this is a
,
                "legitimate, preferred
outcome, not a failure. Do not invent a
"

```

```

        "CLAIM just to produce
output. INFERRED requires a real "
        "falsification_test. You
cannot write to the substrate directly
"
        "-- the gate decides."
    )
    resp =
self.client.chat.completions.create(
    model=self.model,

max_tokens=request.max_tokens,
    messages=[
        {"role": "system",
"content": system_prompt},
        {"role": "user",
"content": (
            f"TASK:
{request.task}\n\nPROJECTED
WORKSPACE:\n"
            f"
{json.dumps(request.projection,
indent=2)}"
        )},
    ],
)
    latency = (time.time() - start)
* 1000
    raw_text =
resp.choices[0].message.content
    candidate =

```

```

_extract_json_candidate(raw_text)
    return ModelResponse(
        raw_text=raw_text,
        model_id=self.model,
        latency_ms=latency,
        token_usage={"input":
resp.usage.prompt_tokens, "output":
resp.usage.completion_tokens},
        candidate=candidate,
    )

class GrokAdapter(ModelAdapter):
    name = "grok"

    def __init__(self, model="grok-4",
api_key=None):
        import openai # xAI's API is
OpenAI-compatible
        self.client = openai.OpenAI(
            api_key=api_key or
os.environ.get("XAI_API_KEY"),

base_url="https://api.x.ai/v1",
        )
        self.model = model

    def generate(self, request:
ModelRequest) -> ModelResponse:
        start = time.time()
        system_prompt = (

```

```

        "You are an operator over a
persistent relational substrate
(PRCSA). "

        "Respond with reasoning,
then a single JSON block: "
        '{"content": "...",
"scope": {...}, "evidence_class":
"REAL|SIMULATED|'

'PREDICTED|INTERVENTION_DERIVED|INFERRE
D", "kind": "CLAIM|HYPOTHESIS|'

'PREDICTION|QUESTION|SIMULATION|ACTION_
PROPOSAL|INTERPRETATION|'

'UNCERTAINTY|TRANSLATION|HOLD",
"falsification_test": null}. '

        "If you do not have enough
support to commit to a claim, respond "
        'with {"kind": "HOLD",
"content": "why"} instead -- this is a
,

        "legitimate, preferred
outcome, not a failure. Do not invent a
"

        "CLAIM just to produce
output. INFERRED requires a real "
        "falsification_test. The
gate decides admission, not you."
    )
    resp =

```

```

self.client.chat.completions.create(
    model=self.model,

    max_tokens=request.max_tokens,
    messages=[
        {"role": "system",
"content": system_prompt},
        {"role": "user",
"content": (
            f"TASK:
{request.task}\n\nPROJECTED
WORKSPACE:\n"
            f"
{json.dumps(request.projection,
indent=2)}"
        )},
    ],
)
latency = (time.time() - start)
* 1000
    raw_text =
resp.choices[0].message.content
    candidate =
_extract_json_candidate(raw_text)
    return ModelResponse(
        raw_text=raw_text,
model_id=self.model,
latency_ms=latency,
        token_usage={"input":
resp.usage.prompt_tokens, "output":
resp.usage.completion_tokens},

```

```

        candidate=candidate,
    )

def _extract_json_candidate(raw_text:
str):
    """Pulls the last JSON object out
of the response text using a
    STRING-AWARE balanced-brace
scanner. The earlier version tracked
    brace depth globally, which broke
on a real, plausible case: a
    model's natural-language content
containing an unmatched brace
    character (e.g. discussing code or
notation). Found by testing
    the actual failure case, not
assumed safe just because a similar
    example happened to work. This
version tracks whether we're
    inside a JSON string literal and
ignores braces while inside one,
    correctly handling escaped quotes
too."""
    candidates = []
    depth = 0
    start_idx = None
    in_string = False
    escape_next = False
    for i, ch in enumerate(raw_text):
        if escape_next:

```

```

        escape_next = False
        continue
    if ch == '\\\' and in_string:
        escape_next = True
        continue
    if ch == '\"':
        in_string = not in_string
        continue
    if in_string:
        continue # braces inside a
string are just characters, not
structure
    if ch == '{':
        if depth == 0:
            start_idx = i
            depth += 1
        elif ch == '}':
            if depth > 0:
                depth -= 1
                if depth == 0 and
start_idx is not None:

candidates.append(raw_text[start_idx:i+
1])

                start_idx = None
    for candidate_text in
reversed(candidates):
        try:
            obj =
json.loads(candidate_text)
        except json.JSONDecodeError:

```

```

        continue
        if obj.get("kind") == "HOLD"
and "content" in obj:
            return obj
            if "content" in obj and
"evidence_class" in obj:
                return obj
            return None

def smoke_test(adapter_class,
**kwargs):
    """The literal first thing to run
once keys exist: reply PONG,
confirm the key/billing/connection
works, before spending anything
on a real PRCSA task."""
    from kernel import ModelRequest
    adapter = adapter_class(**kwargs)
    req = ModelRequest(projection={},
task="Reply with exactly the word PONG
and nothing else.", max_tokens=10)
    resp = adapter.generate(req)
    print(f"[{adapter.name}] raw
response: {resp.raw_text!r}")
    print(f"[{adapter.name}] latency:
{resp.latency_ms:.1f}ms tokens:
{resp.token_usage}")
    return
resp.raw_text.strip().upper() == "PONG"

```

